

ARM PrimeCell Driver

os v1.10 (System Indirection)

API Specification

Primecell driver API description

© Copyright ARM Limited 2000. All rights reserved.

Proprietary notice

ARM, the ARM powered logo, Thumb and StrongARM are registered trademarks of ARM Limited.

The ARM logo, PrimeCell, AMBA, Angel, ARMulator, EmbeddedICE, ModelGen, Multi-ICE, ARM7TDMI, ARM9TDMI, TDMI and STRONG are trademarks of ARM Limited.

All other products or services mentioned herein may be trademarks of their respective owners.

Neither the whole nor any part of the information contained in, or the product described in, this document may be adapted or reproduced in any material form except with the prior written permission of the copyright holder.

The product described in this document is subject to continuous developments and improvements. All particulars of the product and its use contained in this document are given by ARM Limited in good faith. However, all warranties implied or expressed, including but not limited to implied warranties or merchantability, or fitness for purpose, are excluded.

This document is intended only to assist the reader in the use of the product. ARM Limited shall not be liable for any loss or damage arising from the use of any information in this document, or any error or omission in such information, or any incorrect use of the product.

Document confidentiality status

This document is Open Access. This document has no restriction on distribution.

Product status

The information in this document is Final (information on a developed product).

Web address

<http://www.arm.com>

Feedback

ARM limited welcomes feedback on both the product, and the documentation.

Feedback on this document

If you have any comments on about this document, please send email to errata@arm.com giving:

- The document title
- The documents number
- The page number(s) to which your comments refer
- A concise explanation of your comments

General suggestion for additions and improvements are also welcome.

Document Summary

Document Issue	E
Primecell driver identifier	os
Code Version	1.10

Contents

1	ABOUT THIS DOCUMENT	6
1.1	Change control	6
1.1.1	Change history	6
2	SUMMARY	7
3	FEATURES	8
3.1	Internal calls to the Interrupt Controller module	8
3.1.1	The delivered system	8
3.1.2	Interrupt Binding	8
3.1.3	Changing the software	8
3.2	Enabling and disabling interrupts to the processor core	9
3.2.1	The delivered system	9
3.2.2	Changing the software	9
3.3	Internal calls to the timer module	10
3.3.1	Timeout callbacks	10
3.3.2	Delay implementation	10
3.3.3	Changing the software	11
4	DETAILS	12
5	RELEASED FILES	12
6	PUBLIC API	13
6.1	Type Definitions	13
6.1.1	apError	13
6.1.2	apOS_INT_rRawISR	13
6.1.3	apOS_INT_rServiceRoutine	13
6.2	External Functions - Core	14
6.2.1	apOS_CoreFIQDisable	14
6.2.2	apOS_CoreFIQEnable	14
6.2.3	apOS_CoreIRQDisable	14
6.2.4	apOS_CoreIRQEnable	14
6.2.5	apOS_CoreIRQGet	14
6.3	External Functions - Interrupts	15
6.3.1	apOS_INT_ControllerDisable	15
6.3.2	apOS_INT_ControllerRestore	15
6.3.3	apOS_INT_HandlerClear	15
6.3.4	apOS_INT_HandlerSet	15
6.3.5	apOS_INT_InterruptClear	16
6.3.6	apOS_INT_InterruptDisable	16
6.3.7	apOS_INT_InterruptEnable	16
6.3.8	apOS_INT_InstanceGet	16
6.3.9	apOS_INT_SourceGet	16
6.4	External Functions - Timer	17
6.4.1	apOS_TIMER_TimeoutDisable	17

6.4.2	apOS_TIMER_TimeoutEnable	17
6.4.3	apOS_TIMER_Wait	17
6.5	Macros	18
6.5.1	apBIND_ALL_INTERRUPTS	18
6.5.2	apINSTANCE_CHECK	18
6.5.3	apOS_ISR_InterruptDisable	18
6.5.4	apOS_ISR_InterruptEnable	19
6.5.5	apSTATE_GET	20
6.5.6	apVERSION_GET	20
6.5.7	IRQ	20

1 ABOUT THIS DOCUMENT

1.1 Change control

1.1.1 Change history

Issue	Date	Change
A		First issue of API
B	15/11/00	Revised document to match desired template format
C	02/04/01	Updated with addition of 'raw' ISR format
D	18/04/01	Updated in line with release 1.7
E	21/08/01	Updated in line with release 1.10

2 SUMMARY

The system abstraction files abstract the cross-module interactions, operating as a veneer over cross-module calls.

No Primecell driver is permitted directly to invoke functions from another driver.

This is to preclude possible problems relating to OS issues, either due to module availability (paging) or further functionality which is required.

Calls to timers or interrupt controllers are routed to this module.

The system designer may choose to leave the default implementations or to alter these if required.

In general, if a function `apINT_Function()` is required by a module, the matching veneer function in this module will be named `apOS_INT_Function()`.

3 FEATURES

3.1 Internal calls to the Interrupt Controller module

This section describes how the software makes internal calls to the Interrupt Controller module. It contains the following:

3.1.1 The delivered system

The delivered system is preconfigured to use functions named `apOS_INT_*` for internal calls to the Interrupt Controller module.

Each of these is functionally equivalent to the corresponding `apINT_*` function in the Micropack Interrupt Controller module or the corresponding `apVIC_*` function in the PL190 Vectored Interrupt Controller module, according to compilation switches defined in the `apintcfg.h` module and described in the Integration Guide

PL000_DII_1000.

Using these functions provides a hook that can be used to add functionality, such as memory mapping. This makes it easier to enhance the software in future versions.

3.1.2 Interrupt Binding

There are two alternative formats of interrupt binding and associated Interrupt Service Routines (ISRs) supported.

Interrupt binding is always performed by `apOS_INT_HandlerSet`, with prototype below:

```
PUBLIC void apOS_INT_HandlerSet(CONST apOS_INT_oInterruptSource oSource,
                               apOS_INT_rServiceRoutine rHandler,
                               UWORD32 Id,
                               apOS_INT_rRawISR rRawHandler);
```

The interrupt controller must associate (bind) the interrupt source with an ISR. This may be either of:

- ◆ `rHandler` specifies a function of prototype below. The third parameter (`Id`) must be stored by the interrupt controller and passed to the function along with the interrupt source when an interrupt occurs.

```
void MOD_FunctionISR(CONST apOS_INT_oInterruptSource oSource, UWORD32 Instance);
```

- ◆ Alternatively, `rRawHandler` specifies a function of prototype below. This function will be executed directly from the interrupt vector. Since the source and peripheral instance must be identifiable, the two routines below must be implemented to support the raw ISR.

- `apOS_INT_SourceGet` returns the active interrupt source
- `apOS_INT_InstanceGet` returns the identifier passed (as `Id`) to `apOS_INT_HandlerSet`.

3.1.3 Changing the software

The functions are implemented for the Micropack (INT) or PL190 (VIC) interrupt controllers.

If a different interrupt controller is used, function calls to the interrupt controller will have to be linked to support the functionality described.

3.2 Enabling and disabling interrupts to the processor core

This section describes how the software enables and disables interrupts to the processor core. It contains the following information:

3.2.1 The delivered system

The delivered system uses four functions to enable or disable IRQs or FIQs to the processor core:

- ◆ `apOS_CoreIRQDisable`
- ◆ `apOS_CoreIRQEnable`
- ◆ `apOS_CoreFIQDisable`
- ◆ `apOS_CoreFIQEnable`.

Each of these functions is preconfigured to:

1. Read a copy of the CPSR.
 1. Change the value of the I or F control bit in the copy, as appropriate.
 2. Write the changed copy to the CPSR.

You must be in a privileged mode for this write to succeed.

There is also one function to read the current state of IRQs, returning TRUE if IRQs are enabled (core bit not set).

- ◆ `apOS_CoreIRQGet`

These functions are defined in the file `apos\oss.s`.

3.2.2 Changing the software

If you have to call these functions from User mode, you must change their coding appropriately. You must ensure that the processor temporarily enters a privileged mode (such as System mode) so that it can write to the control bits.

3.3 Internal calls to the timer module

This section describes how the software interacts with a timer. The timer may be supplied by ARM or may be a separate driver integrated into the system.

3.3.1 Timeout callbacks

If a module requires a timeout callback, this is implemented through `apOS_TIMER_TimeoutEnable`. This routine specifies a function to be called on elapsing of a specified time. The routine also specifies a timer to use by passing a parameter interpreted as the enumeration of the desired timer.

If the timeout has not elapsed but is no longer required, it can be disabled by a call to `apOS_TIMER_TimeoutDisable` specifying the same timer.

Usage of timeout callbacks is kept to a minimum within the drivers. Where these are required and used, this is documented within the driver API.

Defining as FALSE the constant `apOS_PLATFORM_HAS_TIMERS` in `applatfm.h` will disable this function, so preventing the driver modules from accessing timers.

Note that these routines do not check that the timer(s) used have been initialised – it is the responsibility of the system integrator to initialise all required timers before they are used.

This function is defined in the file `apos\apos.h`.

3.3.2 Delay implementation

When a driver has to wait for a timer, it calls the function `apOS_TIMER_Wait`. This will execute a blocking wait, returning after the specified time. The implementation is as follows:

- ◆ If interrupts are enabled, `apOS_TIMER_TimeoutEnable` is used, with a polling wait for the callback function to be executed. Timer instance 0 will always be used for this function.
- ◆ If interrupts are disabled, or `apOS_TIMER_TimeoutEnable` returns an error (for example if the constant `apOS_PLATFORM_HAS_TIMERS` in `applatfm.h` is defined false), and the C library is available (constant `apOS_CONFIG_USE_NO_CLIB` in `applatfm.h` is not defined false), the C library `clock()` function will be used.
It should be noted that the ARM default implementation of `clock()` is semi-hosted and that the implementation may have poor resolution.
- ◆ If interrupts are disabled, or `apOS_TIMER_TimeoutEnable` returns an error, and the C library is not available, the time delay will be implemented by a counter.
It should be noted that in this case the delay is unlikely to be accurate.

Note that this routine does not check that the timer used has been initialised – it is the responsibility of the system integrator to initialise all required timers before they are used.

This function is defined in the file `apos\apos.h`.

3.3.3 Changing the software

You may wish to modify any of these routines to suit your system configuration and the manner in which your system allocates timers. Suggested changes are:

- ◆ Modified timer API – if a different driver is being used for the timers, `apOS_TIMER_TimeoutEnable` and `apOS_TIMER_TimeoutDisable` can be modified to request the timeout functions according to the driver API.
- ◆ Modified timer allocation – drivers will usually call `apOS_TIMER_TimeoutEnable` with the enumeration of the desired timer as 0 or 1. `apOS_TIMER_Wait` will always pass enumeration 0. The system integrator may choose to assign different timers to the call. For example, if the usage of timer 2 by other system functions is low, this might be used instead.
- ◆ Dynamic timer allocation – if the system can allocate timers according to availability, `apOS_TIMER_TimeoutEnable` might be modified to ignore the enumeration of the desired timer and to allocate a timer at run-time.
- ◆ Improved counter/timer – `apOS_TIMER_Wait` can be implemented by using a counter calibrated specifically for the system used.

4 DETAILS

◆ Peak performance obtained in tests:	N/A
◆ ROM size (RO code + RO data) (release build):	332 Bytes
◆ RAM size (ZI data) per instance (release build):	4 Bytes
◆ Does this driver support both endian-nesses? ¹	YES
◆ Does this driver support Thumb? ²	YES

5 RELEASED FILES

PL000-ISPC-1002-E	This API document
apos/apos.h	Public header file
apos/os.h	Private header file
apos/os.c	Source code
apos/oss.s	Source code

¹ To support both endian-nesses, the driver must be designed to be re-compiled in little-endian or big-endian form.

² The driver may include ARM code, but this must interwork with a Thumb build.

6 PUBLIC API

6.1 Type Definitions

6.1.1 apError

Each module is assigned space for error codes **apERR_XXXX_START** to **apERR_XXXX_START+255**.

Declaration

```
typedef enum apError
```

Elements

apERR_XXXX_START	Start of error codes for module apXXXX
apERR_BAD_PARAMETER	Generic error – Call failed due to a parameter out of range.
apERR_BUSY	Generic error – Resource is already in use
apERR_END	dummy terminator
apERR_NONE	Generic error – no error
apERR_UNSUPPORTED	Generic error – Call failed due to a parameter which was in range, but unsupported for this variant of the hardware.

Remarks

We define an enumerated type **apError**. By using this we avoid the need to specify individual values for the error constants, but can simply append new constants to the list.

Code numbers with the top bit clear are generic errors. Those with the top bit set (negative) are module-specific errors

6.1.2 apOS_INT_rRawISR

This is an alternative format of interrupt service routine, used with a minimal dispatcher and called directly from the interrupt vector.

Declaration

```
typedef IRQ void (*apOS_INT_rRawISR) (void);
```

6.1.3 apOS_INT_rServiceRoutine

This is a generic type for interrupt service routines, called when an interrupt occurs for a particular interrupt source.

Declaration

```
typedef void (*apOS_INT_rServiceRoutine) (CONST apOS_INT_oInterruptSource oSource,  
                                           UWORD32 Parameter)
```

- ◆ the first parameter is the ID of the interrupt source that caused the interrupt.
- ◆ the second parameter is a UWORD32 of data registered with the interrupt handling function. It is used to store the identifier for the peripheral which is handling the interrupt.

6.2 External Functions - Core

6.2.1 apOS_CoreFIQDisable

Disables FIQs to the processor by modifying the processor's program status register.

This function can only be performed in a privileged mode.

Declaration

```
PUBLIC void apOS_CoreFIQDisable(void)
```

6.2.2 apOS_CoreFIQEnable

Enables FIQs to the processor by modifying the processor's program status register.

This function can only be performed in a privileged mode.

Declaration

```
PUBLIC void apOS_CoreFIQEnable(void)
```

6.2.3 apOS_CoreIRQDisable

Disables IRQs to the processor by modifying the processor's program status register.

This function can only be performed in a privileged mode.

Declaration

```
PUBLIC void apOS_CoreIRQDisable(void)
```

6.2.4 apOS_CoreIRQEnable

Enables IRQs to the processor by modifying the processor's program status register.

This function can only be performed in a privileged mode.

Declaration

```
PUBLIC void apOS_CoreIRQEnable(void)
```

6.2.5 apOS_CoreIRQGet

Returns the state of IRQs

Declaration

```
PUBLIC BOOL apOS_CoreIRQGet(void)
```

Return Value

TRUE if IRQs are enabled (core bit not set)

6.3 External Functions - Interrupts

6.3.1 apOS_INT_ControllerDisable

Temporarily disables all interrupts for a particular interrupt controller.

Declaration

```
PUBLIC void apOS_INT_ControllerDisable(apOS_INT_oId oController)
```

Remarks

Disables the interrupt sources for this controller in hardware (keeping a software copy for re-enabling them using apOS_INT_ControllerEnable). All subsequent accesses to the interrupt module act only on this software copy, until such time as the function apOS_INT_ControllerEnable is called to countermand this function. Any other interrupt controllers which funnel into one of the interrupt sources of the controller specified are likely to be disabled by this call.

6.3.2 apOS_INT_ControllerRestore

Use to return to normal operation after a call to apOS_INT_ControllerDisable.

Declaration

```
PUBLIC void apOS_INT_ControllerRestore(apOS_INT_oId oController)
```

Remarks

Re-enables all the interrupts which were enabled before a call to apOS_INT_ControllerDisable. Copies the software version of the interrupt sources which are enabled to the hardware, and allows the other interrupt functions to access the hardware directly again.

6.3.3 apOS_INT_HandlerClear

Removes an interrupt handling function for a specified interrupt source.

Declaration

```
PUBLIC void apOS_INT_HandlerClear(CONST apOS_INT_oInterruptSource oSource)
```

6.3.4 apOS_INT_HandlerSet

Registers an interrupt handling function for a specified interrupt source. Also registers an ID number which will be passed to the interrupt handler when the interrupt occurs.

Declaration

```
PUBLIC void apOS_INT_HandlerSet(CONST apOS_INT_oInterruptSource oSource,  
                                apOS_INT_rServiceRoutine rHandler,  
                                UWORD32 Id , apOS_INT_rRawISR rRawHandler)
```

Parameters

<i>rHandler</i>	the interrupt handling function.
<i>Id</i>	ID number to be passed to the handler function
<i>rRawHandler</i>	alternative interrupt service routine entry point, called directly from the interrupt vector

6.3.5 apOS_INT_InterruptClear

Clears the specified interrupt source.

Declaration

```
PUBLIC void apOS_INT_InterruptClear(CONST apOS_INT_oInterruptSource oSource)
```

6.3.6 apOS_INT_InterruptDisable

Disables the specified interrupt source from producing an interrupt.

Declaration

```
PUBLIC void apOS_INT_InterruptDisable(CONST apOS_INT_oInterruptSource oSource)
```

6.3.7 apOS_INT_InterruptEnable

Enables the specified interrupt source.

Declaration

```
PUBLIC void apOS_INT_InterruptEnable(CONST apOS_INT_oInterruptSource oSource)
```

6.3.8 apOS_INT_InstanceGet

Returns the instance of the peripheral of the currently active interrupt. This routine is called from within the driver raw ISR to obtain the instance of the peripheral requesting service.

Declaration

```
PUBLIC UWORD32 apOS_INT_InstanceGet(void);
```

Return Value

The instance of the peripheral requesting service.

6.3.9 apOS_INT_SourceGet

Returns the interrupt source of the currently active interrupt. This routine is called from within the driver raw ISR to obtain a source to be passed to the interrupt controller routines.

Declaration

```
PUBLIC apOS_INT_oInterruptSource apOS_INT_SourceGet(void);
```

Return Value

The interrupt source of the current interrupt.

6.4 External Functions - Timer

6.4.1 apOS_TIMER_TimeoutDisable

Cancels the timing set up by `apOS_TIMER_TimeoutEnable` so that the callback will not be called. Has no effect if the callback has already occurred.

Declaration

```
PUBLIC void apOS_TIMER_TimeoutDisable(UWORD32 TimerNumber)
```

Parameters

<i>TimerNumber</i>	a specification of which timer is required. How this is interpreted may depend on the implementation.
--------------------	---

6.4.2 apOS_TIMER_TimeoutEnable

Uses the specified timer to call back after a specified delay

Declaration

```
PUBLIC apError apOS_TIMER_TimeoutEnable(UWORD32 TimerNumber,  
                                       UWORD32 Microseconds,  
                                       void rCallback(UWORD32),  
                                       UWORD32 CallbackParam)
```

Parameters

<i>TimerNumber</i>	a specification of which timer is required. How this is interpreted may depend on the implementation.
<i>Microseconds</i>	the length of time to wait in microseconds
<i>rCallback</i>	a callback function called when the timer elapses
<i>CallbackParam</i>	Parameter to be passed to the user callback routine. Usually the identifier for the calling device.

Return Value

- ◆ `apERR_NONE` - timer set successfully
- ◆ `apERR_UNSUPPORTED` - no timers in build (`apOS_PLATFORM_HAS_TIMERS` defined as zero)
- ◆ `apERR_BAD_PARAMETER` - timers

6.4.3 apOS_TIMER_Wait

Uses a timer to wait for a specified length of time (in microseconds). Does not return to the caller until this time span has expired.

Declaration

```
PUBLIC void apOS_TIMER_Wait(UWORD32 Microseconds)
```

Parameters

<i>Microseconds</i>	the length of time to wait in microseconds
---------------------	--

6.5 Macros

6.5.1 apBIND_ALL_INTERRUPTS

*****USED INTERNALLY ONLY*****

Macro to bind all interrupts. A typical invocation is apBIND_ALL_INTERRUPTS(Handler);

Declaration

```
#define apBIND_ALL_INTERRUPTS(_h,_r) {UWORD32 c=Interrupts; if (c) {while (c--) \
    {apOS_INT_HandlerSet(*(pSources+c),_h,oId,_r); \
    apOS_INT_InterruptEnable(*(pSources+c));}}}
```

Remarks

This assumes that the standard naming is used:

- ◆ Interrupts: Number of interrupts
- ◆ pSources: Pointer to array of interrupt sources
- ◆ oId: Identifier for the peripheral

The following parameters are passed:

- ◆ _h – normal handler routine
- ◆ _r – raw interrupt service routine

6.5.2 apINSTANCE_CHECK

*****USED INTERNALLY ONLY*****

This is placed at the start of routines to check that the 'oId' instance parameter is within the allowed range.

This will only apply if apOS_NO_STATIC_STATE is not defined, and so there is a defined range of allowed instance values. Otherwise, we merely assert that the pointer is not NULL.

Declaration

```
#if !apOS_NO_STATIC_STATE
#define apINSTANCE_CHECK(_module) apASSERT(oId<apOS_ ## _module ## _MAXIMUM)
#else
#define apINSTANCE_CHECK(_module) apASSERT(_module)
#endif
```

6.5.3 apOS_ISR_InterruptDisable

Declaration

```
#define apOS_ISR_InterruptDisable(_s) /*apOS_INT_InterruptDisable(_s);*/
```

Remarks

See apOS_ISR_InterruptEnable.

6.5.4 apOS_ISR_InterruptEnable

Declaration

```
#define apOS_ISR_InterruptEnable(_s)    /*apOS_INT_InterruptEnable(_s);*/
```

Remarks

If we have a re-entrant interrupt controller which does not prioritise its interrupts, we must disable an interrupt at the start of the ISR and re-enable at the end, as each ISR is not itself re-entrant. This is not currently the case as the Micropack driver is not re-entrant and the VIC prioritises interrupts.

6.5.5 apSTATE_GET

*****USED INTERNALLY ONLY*****

Data access macro for accessing the state structure for the desired peripheral

Declaration

```
#define apSTATE_GET(_mod, _ptr) (_mod ## _sStateStruct *) (_ptr)
```

Remarks

The data access macro is used to allow the code to be switched between static and heap-based memory. The first parameter is the module name, and the second is an array index (for static memory) or a pointer (for heap memory).

6.5.6 apVERSION_GET

*****USED INTERNALLY ONLY*****

Version access macro used for run-time identification of the Primecell version

Declaration

```
#define apVERSION_GET(_ptr) {UWORD32 * pReg = (UWORD32 *) _ptr->pBase; \
    if \
    ((*(pReg+0x3FC) & 0xFF) | ((*(pReg+0x3FD) & 0xFF) << 8) | ((*(pReg+0x3FE) & 0xFF) << 16) | ((*(pReg+ \
    0x3FF) & 0xFF) << 24) == 0xB105F00D) \
    { _ptr->Version = (*(pReg+0x3F8) & 0xF) + ((*(pReg+0x3F8) & 0xF0) >> 4) * 10 + \
    (*(pReg+0x3F9) & 0xF) * 100; } \
    else { _ptr->Version = 0; }}
```

Remarks

This macro loads the identifier from the Primecell registers into the Version element of the passed structure. If these are absent, the version is set to zero.

The parameter is a pointer to the state data for the driver, and it is assumed that this has the elements below:

- ◆ pBase - The register base address
- ◆ Version - an element to store the version identifier

6.5.7 IRQ

Defines a function to be an ISR (Interrupt Service Routine) called directly from the IRQ or FIQ vector.

Declaration

```
#ifdef __thumb
    #define IRQ
#else
    #define IRQ __irq
#endif
```

Note

A function compiled as Thumb code cannot be an ISR called directly from the IRQ or FIQ vector. For this reason, the keyword is ignored when compiling as Thumb.